Analysis of Architectural Design Choices for Brain-to-Text Decoding

Sean Shen

An essay submitted to the faculty at the University of North Carolina at Chapel Hill in partial fulfillment of the requirements for the degree of Master of Science in the Department of Statistics and Operations Research.

Chapel Hill
2026

Approved by:

Professor Yao Li
Professor Glenn Sabin
Professor Zhengwu Zhang
Professor Chudi Zhong

ABSTRACT

Sean Shen: Analysis of Architectural Design Choices for Brain-to-Text Decoding
(Under the direction of Professor Yao Li and Professor Glenn Sabin)

Advances in brain-computer interfaces demonstrate the potential in restoring communication by decoding neural activity into text. This paper systematically investigates the specific architectural components of an intracortical speech decoding pipeline with a recurrent neural network backbone, focusing on the transition from high-dimensional neural signals to phoneme predictions. Findings show that the temporal compression is the most critical component in the pipeline, as it allows the model to perceive neural trajectories and better model the relationship between neural activity and phonetic intent. Ablations of model depth and width also revealed minor improvements that could be achieved through increasing the recurrent capacity. Compounding those findings and configurations, the final testing Phoneme Error Rate achieved is 9.68%.

TABLE OF CONTENTS

1 INTRODUCTION

1.1 Motivation for Brain to Text Decoding

Brain-to-text decoding is the process of translating high-dimensional neural activity from the motor cortex into discrete linguistic units using machine learning. It functions by mapping the patterns of electrode voltages associated with intended speech directly into a sequence of phonemes or text. The motivation for brain-to-text decoding is to restore communication to people who lost their ability to speak due to conditions such as Amyotrophic Lateral Sclerosis (ALS) or brainstem strokes. For those people, the distance between intended thought and external expression hinders a good quality of life. Recent advancements in Brain-Computer Interfaces (BCIs) close this distance by directly translating neural signals into text.

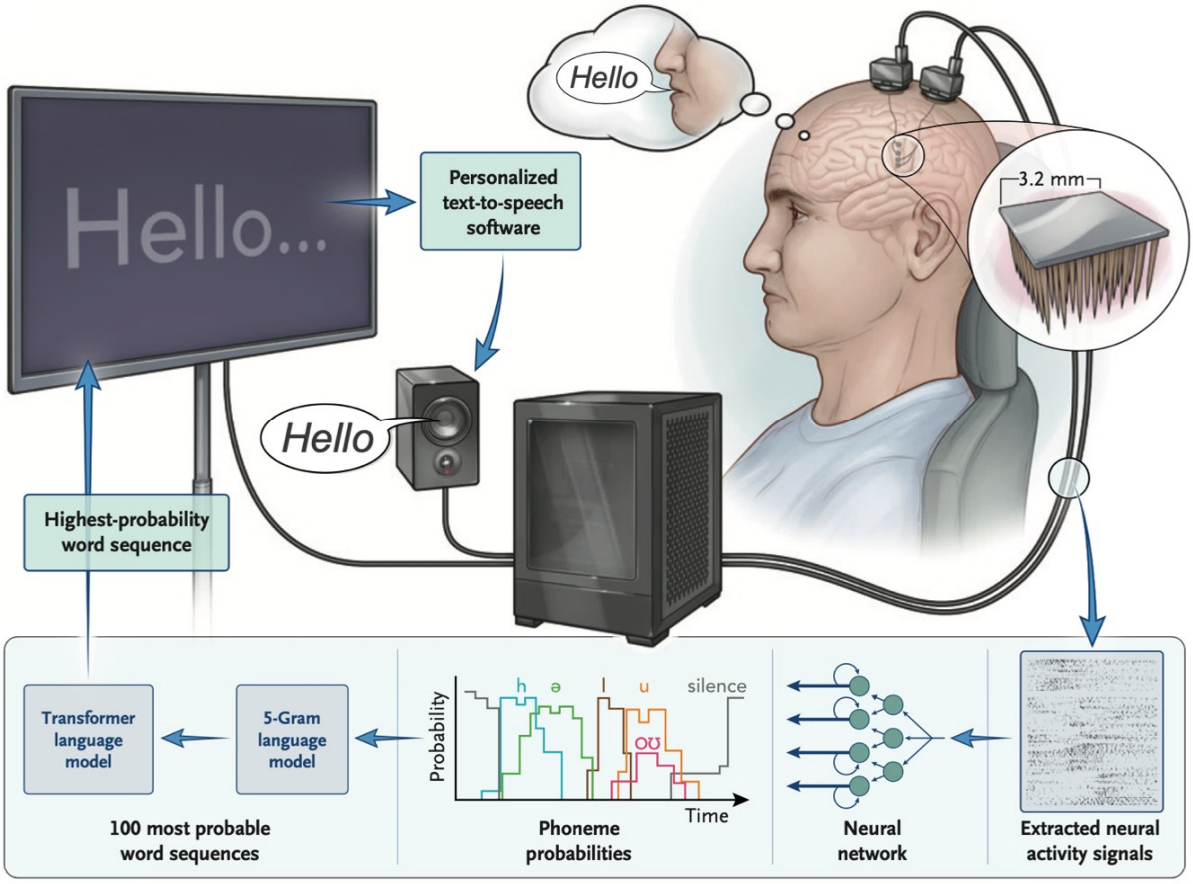

Figure 1: The full brain-to-text decoding pipeline (Card et al.[3]), including neural signal extraction, recurrent neural network processing, and language model integration. This paper focuses specifically on optimizing the neural network component responsible for generating phoneme probabilities.

Researchers from UC Davis developed a BCI [3] using deep learning methods that achieved 97.5% accuracy for decoding the attempted speech of a man with ALS and severe dysarthria using intracortical neural data. Successes such as these show promising progress to clinical translation. However, there is still a long way to go before a reliable real-time communication device is available, because of the inherent complexity of neural data and the black-box nature of neural networks. This paper utilizes ablation studies to break down the specific architectural choices that contribute to the success of UC Davis' Brain-to-Text system, and specifically the component responsible for decoding neural signals into phoneme sequences. Ablation studies are a systematic experimental method used to evaluate the contribution of individual components within a complex system by removing or modifying them one at a time. To provide a foundation, the paper will first detail the characteristics of the high-dimensional neural data and then dissect the recurrent neural network architecture.

2 NEURAL DATA

2.1 Data Acquisition and Structure

The dataset used in the study contains neural recordings from a forty-five-year-old male participant with ALS and severe dysarthria spanning forty-five sessions over a twenty-month period, comprising 10,948 spoken sentences [3]. There are many trials in one session, each of which corresponds to a single spoken sentence. Each trial contains 3 distinct functional epochs:

- Delay Epoch: A trial begins with the delay epoch, where the sentence prompt for speaking is displayed on the screen, but the participant is not allowed to speak yet. This allows a baseline to validate the specificity of the speech detection algorithm, thus depicting whether the model can successfully distinguish between a resting state and the intention to speak with a false positive rate of less than 1%.

- Go Epoch: This is followed by the go epoch, where the participant attempts to speak, and the speech motor cortex produces the electrical fluctuations required for decoding.

- Post-Trial Epoch: The post-trial epoch is the period after the attempted speech, listening to the synthesized audio of the decoded sentence.

The trials are organized in blocks. Each block has around 50 uninterrupted trials, lasting around 15 to 25 minutes. For each trial, four high-density microelectrode arrays (64 electrodes each) were surgically installed into the speech motor cortex(left precentral gyrus) to record the continuous voltage changes at a sampling rate of 30 kHz.

Those raw neural readings are then segmented into 1 millisecond windows where a band-pass filter (4th order Butterworth filter) and a denoising technique called linear regression referencing are applied to each of the windows [3]. The band-pass filter isolates the fluctuations in the band of 250 Hz to 5000 Hz, capturing the electric signals of the neurons firing while removing the noises from the other frequencies that are not from the brain, such as the background electronic static. The linear regression referencing subtracts global electrical interference from the signals such that the remaining data represents the neural signals from the brain. Once the signals are filtered and denoised, the system extracts two primary features from each of the 256 electrodes. The -4.5 RMS (root mean square) threshold crossings measure how many times spikes hit a specific voltage limit, indicating how many times neurons have fired. The spike band power captures the total energy or intensity of the electrical activity. Those two metrics are commonly used for detecting neuron spiking activities.

Because a single millisecond is too short and noisy for a recurrent neural network to reliably interpret, those signals or recorded values are binned at a temporal resolution of 20 milliseconds. So each time step is 20 milliseconds, containing 512 unique features. For a single trial, the data looks like a 512 $\times$ T matrix, with T denoting the number of time steps or 20 millisecond bins[3].

Before those 512-value vectors are sent to a neural network, they are dynamically normalized using rolling Z-scoring. Rather than establishing a single, fixed baseline for the participant's brain activity, a sliding window was used for calculating the mean and standard deviation for each of the 512 features exclusively from the go epochs of the 20 most recent trials, then standardizing each of the features with their respective mean and standard deviations. Rolling Z-scoring constantly updates the system's baseline to adapt to natural, ongoing drifts in the participant's brain activity throughout the day. After normalizing to Z-scores, a Gaussian kernel with a standard deviation of 40 milliseconds is applied to smooth out erratic noises and spikes in the feature vectors to provide a more stable representation of the participant's neural activity for decoding.

Sentences are represented by phonemes, which are the smallest discrete units of sound or articulation that combine to form words. The goal of the recurrent neural network is to predict sequences of

phonemes that represent the sentences that the participant attempted to speak. The dataset contains the ground truth sentence labels and the ground truth phoneme sequence labels with respect to each of the 512 $\times$ T matrices representing the neural data.

```
Session: t15.2023.09.01, Block: 9, Trial: 20
Sentence label:     He's just goofing off like he always has.
True sequence:      HH IY Z  |  JH AH S T  |  G UW F IH NG  |  AO F  |  L AY K  |  HH IY  |  AO L W EY Z  |  HH AE Z  |
Predicted Sequence: HH IY Z  |  JH AH S T  |  G UW V IH NG  |  AO F  |  L AY K  |  HH IY  |  AO L W EY Z  |  HH AE Z  |
Phoneme Error rate: 2.86%

Session: t15.2023.09.01, Block: 8, Trial: 6
Sentence label:     That's what they said.
True sequence:      DH AE T S  |  W AH T  |  DH EY  |  S EH D  |
Predicted Sequence: DH AE T S  |  W AH T  |  DH EY  |  S EH D  |
Phoneme Error rate: 0.00%

Session: t15.2023.09.01, Block: 9, Trial: 19
Sentence label:     They live with us.
True sequence:      DH EY  |  L AY V  |  W IH DH  |  AH S  |
Predicted Sequence: DH EY  |  L AY V  |  W IH DH  |  AH S  |
Phoneme Error rate: 0.00%

Session: t15.2023.09.01, Block: 8, Trial: 17
Sentence label:     She didn't announce that to you.
True sequence:      SH IY  |  D IH D AH N T  |  AH N AW N S  |  DH AE T  |  T UW  |  Y UW  |
Predicted Sequence: SH IY  |  D IH D AH N T  |  AH N AW N T S  |  DH AE T  |  T UW  |  Y UW  |
Phoneme Error rate: 3.85%

Session: t15.2023.09.01, Block: 9, Trial: 11
Sentence label:     My name is Pat Johnson and I live in Texas.
True sequence:      M AY  |  N EY M  |  IH Z  |  P AE T  |  JH AA N S AH N  |  AH N D  |  AY  |  L AY V  |  IH N  |  T EH K S AH S  |
Predicted Sequence: M AY  |  N EY M  |  IH Z  |  P AE T  |  CH AH N S AH N  |  AH N D  |  AY  |  L AY V  |  IH N  |  T EH K S AH S  |
Phoneme Error rate: 4.88%
```

Figure 2: "True Sequence" is the ground-truth label. The model generates a "Predicted Sequence" to match a ground-truth label.

In this context, ground truth refers to the intended phoneme sequence that the participant was prompted to speak. Because the participant is unable to produce audible speech, these labels serve as the objective correct target that the model attempts to replicate. The phoneme sequence labels consist of 39 distinct phonemes, a silence token "|", and a "blank" token. The ground truth phoneme sequence labels are used to evaluate the predictions made by the neural network.

3 METHODOLOGY

3.1 Architecture Overview

The experimental design and data processing pipeline described in this section are a summary of the framework established by Card et al.[3]. The following formulation provides the baseline context for the ablation studies presented in this paper.

The objective of the BCI developed by Card et al. [3] is to map continuous neural features to discrete probability distributions over a defined phoneme vocabulary. The architecture to achieve this utilizes a

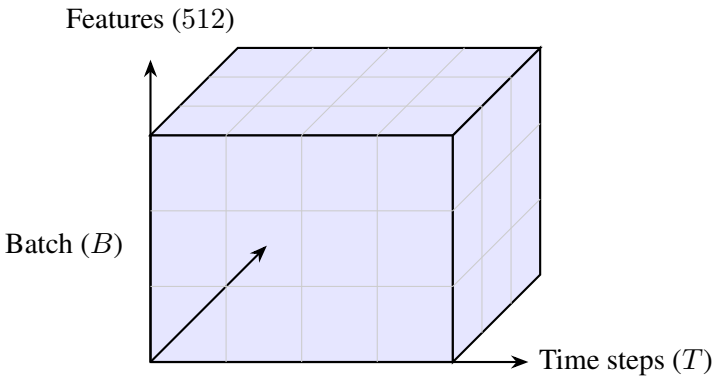

Figure 3: Tensor structure of a batch of neural input data, with B trials in the batch. Each trial has T time steps. Each time step has 512 neural features.

recurrent neural network as its backbone to handle the sequential nature of neural recordings and speech production.

The model receives an input tensor $X \in \mathbb{R}^{B \times T \times D}$, where B is the batch size, T is the number of time steps, and D is the number of neural features. There are four distinct computational stages in the forward pass:

- the day alignment, which accounts for the non-stationarity between the different days/sessions caused by the installation of electrodes,
- temporal compression, which captures long-term dependencies within the neural signals,
- the gated recurrent unit (GRU), which captures the temporal and the phonetic context inside the neural signals,
- and the linear classifier, which maps the recurrent hidden states from the GRU into the phoneme vocabulary space and produces predictions.

The following figure outlines the entire brain-to-text architecture.

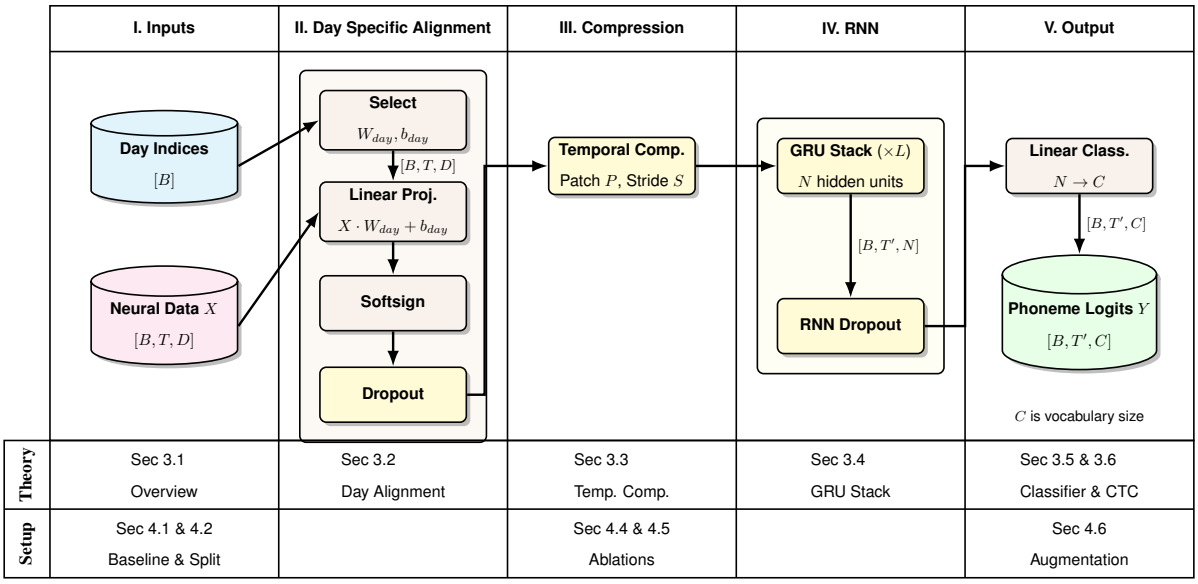

Figure 4: The pipeline for brain-to-text decoding. The architecture processes raw neural data X and session indices through linear day alignment, temporal compression, a multilayer GRU, and a final linear classifier to produce phoneme logits Y.

3.2 Day Alignment

Intracortical neural data suffer from non-stationarity across different recording sessions due to the micro shifts in the placement of the electrode array. The day alignment addresses the non-stationarity by projecting the neural features from the different sessions or recording days into a single shared latent space. The layer receives an input tensor $X \in \mathbb{R}^{B \times T \times D}$, where B is the batch size, T is the number of time steps that will be dynamically determined using the longest trial of that batch, and D is the number of neural features. For each trial in the batch, there is a corresponding session index $d \in \{1, 2, \ldots, N\}$, denoting the specific session the recording occurred in out of N total sessions. Let b be the index of a trial among the B trials in a batch ($b \in \{1, 2, \ldots, B\}$), t be the time step within that trial, and d_b be the recording session of that trial b. Then the network uses a weight matrix $W_d \in \mathbb{R}^{D \times D}$ and a bias vector $b_d \in \mathbb{R}^D$ for every single recording session d to correct for the non-stationarity. For a batch, the network identifies the recording session index d_b of trial b in the batch. The neural data vector for trial b is transformed according to the following equation:

$$Z_{b,t} = X_{b,t} W_{d_b} + b_{d_b} \tag{1}$$

where $X_{b,t} \in \mathbb{R}^{1\times D}$, $W_{d_b} \in \mathbb{R}^{D\times D}$, $b_{d_b} \in \mathbb{R}^{1\times D}$, and the resulting projected vector is $Z_{b,t} \in \mathbb{R}^{1\times D}$. Every W_d is initialized as an identity matrix, and b_d is initialized as a zero vector. Their values will be changed by the backpropagation of the loss function. By doing this calculation in equation (1) for every individual time step t across all trials b, the tensor $Z \in \mathbb{R}^{B\times T\times D}$ is constructed.

To introduce nonlinearity, Z passes through a softsign activation function:

$$A = \frac{Z}{1 + |Z|} \tag{2}$$

where $A \in \mathbb{R}^{B\times T\times D}$ contains values between -1 and 1 preventing exploding gradient. A is the scaled counterpart of the original batch of neural vectors aligned by the differences between the recording sessions. To prevent the neural network from over relying on any single neural feature, A undergoes an inverted dropout where a mask M with the exact dimensions of A is applied through element-wise multiplication. Each independent element within this mask is drawn from a Bernoulli distribution by a dropout probability p: $M_{b,t,d} \sim \text{Bernoulli}(1-p)$. Because randomly dropping a percentage of the signals decreases the magnitude of the tensor, this will cause a discrepancy during evaluation since dropout will be disabled. Then, the surviving values in A will be scaled upward by the dropout probability as follows:

$$\tilde{X} = (A \odot M)\frac{1}{1-p} \tag{3}$$

where $\odot$ is element-wise multiplication and $\tilde{X} \in \mathbb{R}^{B\times T\times D}$.

As mentioned before, T is the number of time steps for a batch X dictated by the longest trial within that batch. Naturally, there would be padded frames in some of those trials that get transformed into non-zero values because of the matrix multiplication and the addition of the bias vector. To combat the bias introduced by the transformation, the unpadded length of each trial is preserved and passed to the CTC loss function to mask out the padded frames during the final calculation. The implementation of the CTC loss function is detailed in Section 3.6.

3.3 Temporal Compression

The day alignment layer produces a standardized input tensor $\tilde{X} \in \mathbb{R}^{B\times T\times D}$(3), which is sampled at a very granular resolution of 20 milliseconds per time step. The mechanism begins with $\tilde{X} \in \mathbb{R}^{B\times T\times D}$

from the day alignment step. The model applies a sliding window operation across the temporal dimension T, which is dictated by the patch size P and the stride S. The patch size determines the number of consecutive time steps to be included in a single window. The stride dictates the number of time steps it moves forward after taking a window. The patches specifically in this paper are not causal for offline training. This patch and stride mechanism is also known as temporal compression.

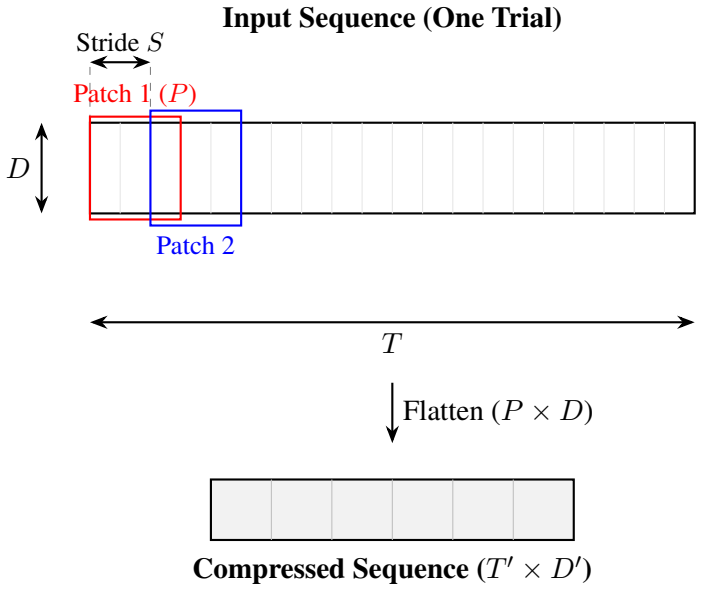

Figure 5: Mechanism of temporal patching. A sliding window of size P extracts neural features from the $T \times D$ input with a stride of S. These are flattened into vectors of length D', resulting in a reduced sequence length T'.

When the window extracts these P frames, it creates a block of data with dimensions $P \times D$, which will be flattened into a vector of length $D' = P \times D$. The flattening works by reading all dimensions of a time step, then moving on to the next time step. Since the window skips forward by S steps for each patch, the overall length of the sequence will be compressed. The new sequence length will be

$$T' = \left\lfloor \frac{T - P}{S} \right\rfloor + 1 \tag{4}$$

The final vector after temporal compression will be $U \in \mathbb{R}^{B \times T' \times D'}$. By providing a window of context, the RNN that follows will not have to learn the micro-fluctuations between the frames and instead learn the underlying temporal relationships between neural activities and speech.

3.4 Gated Recurrent Unit

The temporal compression step produces $U \in \mathbb{R}^{B \times T' \times D'}$ that includes the contextual information for each time step. The architecture then uses a multilayer unidirectional Gated Recurrent Unit (GRU). Those recurrent layers are responsible for analyzing the local observations provided by the patches and tracking the temporal patterns of the brain signals.

Let $U_t \in \mathbb{R}^{D'}$ denote the input vector for a single trial at time step t. The GRU maintains a hidden state vector $h_t \in \mathbb{R}^N$ of size N where N is the number of hidden units. The hidden state vector is sequentially updated to encode the temporal history of that specific trial. There are two gates responsible for controlling the information into and out of this hidden state: the update gate z_t and the reset gate r_t. The update gate determines how much of the past memory will be carried forward. The reset gate decides how much of the past memory will be forgotten. Both the reset gate and update gate go through a sigmoid [1] activation. The activation layer allows the neural network to model nonlinear relationships. The flow of information in and out of the hidden states is defined in the following equations[4]:

$$r_t = \sigma(W_{ir}U_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

$$z_t = \sigma(W_{iz}U_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

$$\tilde{h}_t = \tanh(W_{in}U_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn}))$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

where W_{xy} and $b_{xy}, x \in \{i, h\}, y \in \{r, z, n\}$ [2] are weight matrices and bias vectors, respectively, that map from x to y. The letters i and h represent the input vectors and hidden state vectors, respectively. r, z, and n represent the reset gate, update gate, and candidate hidden state, respectively. Then W_{ir} would represent the weight matrix to map the input data vector to reset the gate, resizing the matrix and translating the neural signals to phonetic logic. $\tilde{h}_t$ is the candidate hidden state used for updating the h_{t-1} to form the new hidden state h_t. The input weights (W_{ir}, W_{iz}, W_{in}) are initialized using a Xavier uniform distribution [3], while the recurrent hidden weights (W_{hr}, W_{hz}, W_{hn}) uses an orthogonal

[1] Sigmoid activation: $\sigma(x) = \frac{1}{1+e^{-x}}$

[2] $W_{ir}, W_{iz}, W_{in} \in \mathbb{R}^{N \times D'}, W_{hr}, W_{hz}, W_{hn} \in \mathbb{R}^{N \times N}, b_{ir}, b_{iz}, b_{in}, b_{hr}, b_{hz}, b_{hn} \in \mathbb{R}^N$

[3] $Var(Output) = Var(Input), E[W] = 0, Var(W) = \frac{2}{D'+3N}$

initialization [4] to prevent the gradients from vanishing or exploding during backpropagation. The initial hidden state h_0 is instantiated as a learnable parameter initialized via a Xavier uniform distribution once before training.

The first recurrent layer takes in $U \in \mathbb{R}^{B \times T' \times D'}$ and outputs the first hidden state sequence $H^{(1)} \in \mathbb{R}^{B \times T' \times N}$. If there are subsequent GRU layers stacked afterwards, $H^{(1)}$ will be the input and $H^{(n)} \in \mathbb{R}^{B \times T' \times N}$ will be the n-th hidden state sequence. $H^{(n)}$ encapsulates the temporal history of the neural signal, distilling the linguistic intent of the participant.

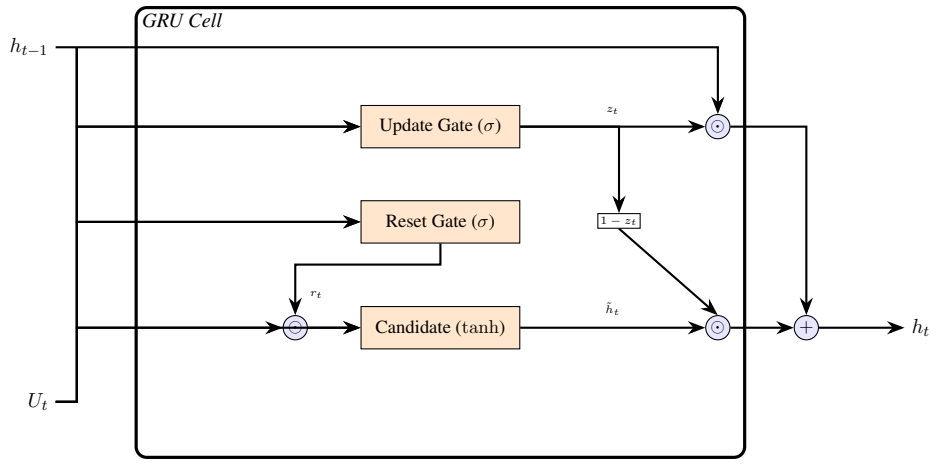

Figure 6: Internal architecture of the Gated Recurrent Unit. The update gate z_t and reset gate r_t modulate the flow of information from the previous hidden state h_{t-1} and the current compressed neural input U_t to produce the new state h_t.

3.5 Linear Classifier

After the processing of input neural data through stacked GRU layers, converting the data into a linearly separable format as a product of optimization, the resulting hidden state sequences will be turned into phonetic predictions. $H^{(n)} \in \mathbb{R}^{B \times T' \times N}$ is still confined to the hidden unit dimension N. Those hidden state sequences are projected onto the predefined phoneme vocabulary.

A dense feedforward linear classification layer is used to project the latent representation into predictions. For every time step $t \in T'$, the hidden state vector $h_t \in \mathbb{R}^N$ is passed through this layer with no nonlinear activation function, defined by the following equation:

$$y_t = W_{out} h_t + b_{out}$$

[4] $W^T W = I$

where $y_t \in \mathbb{R}^C$ represents the output logit vector for time step t, with C denoting the total number (41) of target classes. The learned parameters for this layer are the output weight matrix $W_{out} \in \mathbb{R}^{C \times N}$ and the output bias vector $b_{out} \in \mathbb{R}^C$. $W_{out} \in \mathbb{R}^{C \times N}$ is initialized using a Xavier uniform distribution for gradient stability during optimization. After this operation, the final matrix of unnormalized predictions $Y \in \mathbb{R}^{B \times T' \times C}$ is produced. Finally, the raw logits are passed directly into the CTC loss function for evaluation and to calculate the most probable sequence of phonemes. The implementation of the CTC loss function is detailed in Section 3.6.

3.6 Connectionist Temporal Classification

One challenging aspect of decoding neural signals into text is that there is no alignment between the continuous biological input and those discrete phoneme labels, as it is unclear which part of the continuous neural signals corresponds to which phonemes. Also, because the input neural features are significantly longer than the output sequences of phonemes, the exact timing of each phoneme is unknown. Connectionist Temporal Classification (CTC) loss is an objective function that addresses this many-to-one mapping problem by summing the probabilities of all of the possible alignments that collapse into the target sequence, introduced by Graves et al.[5]. An alignment is a specific path of predicted phonemes across all time steps, which, when collapsed, results in a sequence. This mechanism allows the model to be trained without needing to create time-aligned labels for each individual phoneme.

The recurrent neural network (RNN) in the BCI [3] outputs a probability distribution of all 41 phonemes at each time step of prediction. Any alignments that result in the correct sequence are valid alignments. The joint probabilities of all valid alignments will be summed together as the probability of predicting the correct sequence. To avoid the computationally impossible task of calculating the probabilities of every possible incorrect alignment, the CTC algorithm utilizes dynamic programming (forward-backward algorithm) to efficiently aggregate only the valid paths. Finally, the CTC loss is calculated using cross-entropy on the ideal distribution, where the probability of the correct sequence being 1 and its complement being 0.

Let $Y \in \mathbb{R}^{B \times T' \times C}$ be the matrix of unnormalized logits output from the linear classifier. To convert these raw logits into valid probability distributions over the C phoneme classes, a softmax function is applied at each time step $t \in T'$. The probability $s^t_{b,k}$ of observing class k (predicting phoneme k) at

time step t for trial $b \in \{1, 2, \ldots, B\}$ is calculated as:

$$s_{b,k}^t = \frac{\exp(y_{b,k}^t)}{\sum_{j=1}^{C} \exp(y_{b,j}^t)} \tag{5}$$

where $y_{b,k}^t$ is the unnormalized logit for class k at time step t within trial b.

Let:

- S_b be the ground truth target phoneme sequence of trial b.
- π be a single valid alignment path of length T'.
- $\Phi(S_b)$ be the set of all valid paths that map to the correct sequence S_b, where s_{b,π_t}^t is the predicted probability of the specific phoneme π_t at time step t for trial b.

The probability of the correct sequence S_b given the model output Y_b for that specific trial is defined as the sum of the probabilities of all valid alignment paths:

$$P(S_b|Y_b) = \sum_{\pi \in \Phi(S_b)} P(\pi|Y_b) \tag{6}$$

The probability of a single path π is the product of the probabilities of the predicted phonemes across all compressed time steps T':

$$P(\pi|Y_b) = \prod_{t=1}^{T'} s_{b,\pi_t}^t \tag{7}$$

The CTC loss for an individual trial b is then the negative log likelihood of the correct sequence:

$$\mathcal{L}_{CTC}^{(b)} = -\log \left(\sum_{\pi \in \Phi(S_b)} \prod_{t=1}^{T'} s_{b,\pi_t}^t \right) \tag{8}$$

During training, the linear classifier outputs a batch of size B. The final objective function computes the individual loss for each trial and averages them to produce the overarching batch loss used for backpropagation:

$$\mathcal{L}_{batch} = \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}_{CTC}^{(b)} \tag{9}$$

To understand this loss function intuitively, please see Appendix 1 for a toy example.

The final step in evaluating the performance of the recurrent neural network is the phoneme error rate (PER). This metric is calculated by comparing the predicted sequence produced by the RNN to the ground truth phoneme label using Levenshtein distance, an algorithm that computes the minimum number of single phoneme edits required to transform the predicted sequence into the ground truth sequence.

Let N represent the total number of phonemes present in the ground truth sequence. The Levenshtein algorithm identifies three distinct types of sequence errors:

- Substitutions (S): Instances where the model predicts an incorrect phoneme in place of the correct ground truth phoneme.
- Deletions (D): Instances where a phoneme present in the ground truth sequence is entirely omitted from the predicted sequence.
- Insertions (I): Instances where the model predicts an extraneous phoneme that does not correspond to any target unit in the ground truth sequence.

The Phoneme Error Rate is formally defined as the total sum of all required editing operations divided by the length of the ground truth sequence:

$$PER = \frac{I + D + S}{N}$$

4 EXPERIMENTAL SETUP AND RESULTS

4.1 Baseline Parameters

The translation model from the BCI[3] consists of 5 stacked GRU layers, each with 768 units. A dropout rate of 0.4 is applied to the recurrent connections, reducing overfitting and forcing the network to learn the representations of speech rather than memorizing the noise in the neural data. The model is trained using the Adam optimizer with a total budget of 120,000 batches. The learning rate follows a cosine decay schedule, starting out with a 1,000-step linear warm-up to a max learning rate of 0.005 before gradually decaying to a minimum of 0.0001. This schedule allows the model to explore the loss

landscape aggressively early and close in on a convergence. Gradient norm clipping is set to a value of 10 to prevent exploding gradients. For temporal compression, the model uses a patch size of 14 and a stride of 4, which corresponds to a window size of 280 ms of neural activity, and makes a prediction every 80 ms.

Given the inherently noisy nature of neural data, several data augmentation techniques are used to improve generalization. The following stochastic transformations are applied to the input sequences during training:

- Additive white noise: During training, additive white noise sampled from $\mathcal{N}(0, \sigma^2)$ with a standard deviation of 1.0 is injected into the input tensor. The resulting noise tensor has a shape of $\mathbb{R}^{B \times T \times D}$.

- Constant offset noise: For each trial in a batch, constant offset noise sampled from a Gaussian distribution with a standard deviation of 0.2 generates a noise tensor of shape $\mathbb{R}^{B \times 1 \times D}$. This noise tensor is broadcasted across the time dimension T for each trial in the batch.

- Random cut: The model randomly removes up to 3 time steps [5] of data at the beginning of each trial to vary the start time of the participant speaking because of the delay epoch.

- Gaussian smoothing: The model then applies a one-dimensional Gaussian kernel of size 100 and a standard deviation of 2.0, convolved along the time axis with a stride of 1.

4.2 Data Split

The data provided by the Brain-to-text 2025 [2] contains a train, validation, and test set for most of the sessions. However, not all of the test sets have ground truth labels for the phoneme sequences. Therefore, the original validation set was split into a new validation set and a new test set, where the test set will never be used until the very last evaluation. The data splitting mechanism uses a seed to ensure that the same test set is sampled for any configurations and on any training runs, to ensure that any improvements in PER are due to the model architectural changes or hyperparameter adjustments rather than a different data split.

[5] Random cut follows a discrete uniform distribution over {0, 1, 2, 3} time steps removed

4.3 Training

Training is executed on the UNC Longleaf cluster using an NVIDIA Volta GPU and 8 CPU cores. The training routine is set to a total budget of 120,000 batches for the learning rate scheduler, though typically a configuration reaches convergence at approximately 65,000 batches trained. Each batch consists of 64 trials sampled from 4 distinct sessions to ensure the model gradients get updated by different neural distributions and prevent the model from overfitting to any specific day with more data. For each of the 4 sessions in a batch, the trials are sampled with replacement.

The Adam optimizer has a weight decay of 0.001 and gradient norm clipping set to 10 for stable updates. Because multiple configurations might take longer to converge, requiring more than one job submission on Longleaf, the trainer saves a state dictionary for the best checkpoint in a training run. The state dictionary preserves the model parameters, the optimizer state, and the learning rate scheduler state, so that the same configuration can be continued with another job submission. The learning rate follows a cosine decay schedule with a 1,000-step linear warm-up. After reaching a maximum of 0.005, the rate decays toward a floor of 0.0001 over the 120,000 batches. Validation occurs across 2,000 batches.

4.4 Importance of Temporal Compression

The first phase of the experiments focuses on identifying the components in the architecture that contribute most significantly to lowering the phoneme error rate. By modifying the depth, the width, and the temporal processing layers, the features most essential for model performance were isolated. The baseline model, introduced in section 4.1, features a 5 GRU stack layers, 768 hidden units, a patch size of 14, a stride of 4, a RNN dropout of 0.4, and a day alignment dropout of 0.2.

The convergent validation Phoneme Error Rates (PER) for each architectural configuration are compared against the baseline model in the table below. The configurations describe the specific parts of the architecture that are being ablated to study their effects. The "Configuration" column in each table corresponds to the stages defined in the overall architecture diagram (Figure 4). Specifically:

- **Temporal Compression:** Corresponds to Stage III (Compression). This ablation evaluates the temporal downsampling mechanism by varying the sequence patch size (P) and stride size (S).
- **Model Depth:** Corresponds to Stage IV (RNN). This ablation investigates the vertical scale of the sequence model by modifying the number of stacked GRU layers (L).

- **Model Width:** Corresponds to Stage IV (RNN). This ablation tests the representational capacity of the network by varying the number of hidden units (N) within the GRU stack.
- **Regularization:** Corresponds to the Dropout operations in Stage II (Day Specific Alignment) and Stage IV (RNN). This experiment assesses the model reliance on stochastic regularization by disabling all dropout layers during training.
- **Data Augmentation:** Corresponds to Stage I (Inputs). Because transformations are applied stochastically to the Neural Data (X) prior to the forward pass, this ablation isolates the impact of the artificial augmentation suite on model generalization.

Configuration	**Modification**	**Convergent PER**	**Δ from Baseline**
Baseline	Baseline	0.1054	0.0000
Temporal Compression	No Patching (Raw Sequences)	0.2204	+0.1150
Model Depth	GRU (2 Layers)	0.1219	+0.0165
	GRU (1 Layer)	0.1668	+0.0614
Model Width	512 Hidden Units	0.1114	+0.0060
	1024 Hidden Units	0.1038	-0.0016
Regularization	No Dropout	0.1090	+0.0036
Data Augmentation	No Data Augmentation	0.1832	+0.0778

The most significant performance degradation occurred when the patching mechanism was removed, resulting in a PER increase from 0.1054 to 0.2204. This observation shows that patching, or temporal compression, had the highest reduction in the validation phoneme error rate.

The patching layer concatenates a window of time steps into a high-dimensional vector, reducing the sequence length T that the Gated Recurrent Unit (GRU) must process. By aggregating localized temporal information before it enters the recurrent layers, the model can focus on learning the long-range phonetic and temporal dependencies instead of learning the noise between the neighboring frames.

While width and depth changes did influence the PER, their impacts are much smaller than the patching mechanism. Reducing the model from 5 layers to 1 layer increased the PER to 0.1668, indicating that a multi-layer hierarchy is needed for extracting a more abstract understanding of the data. Increasing the hidden units to 1024 yielded the lowest overall PER of 0.1038. While decreasing the

hidden units to 512 slightly increased the PER to 0.1114. Disabling the dropout in the RNN and the day layer increased the PER to 0.1090, indicating that it is effective for preventing overfitting on the training data.

4.5 Temporal Compression / Patch and Stride Ablations

Configuration	Modification	Convergent PER	Δ from Baseline
Baseline	Patch 14, Stride 4	0.1054	0.0000
Temporal Compression	P 22, Stride 4 ($+8$ P)	0.1042	-0.0012
Temporal Compression	Patch 14, Stride 3 (-1 S)	0.1059	$+0.0005$
Temporal Compression	Patch 14, Stride 2 (-2 S)	0.1054	0.0000
Temporal Compression	Patch 10, Stride 4 (-4 P)	0.1097	$+0.0043$
Temporal Compression	Patch 6, Stride 4 (-8 P)	0.1117	$+0.0063$
Temporal Compression	Patch 2, Stride 1 (-12 P -3 S)	0.1077	$+0.0023$
Temporal Compression	No Patching (Raw Sequences)	0.2204	$+0.1150$

Table 1: The convergent validation PER for temporal compression/patch and stride ablations are compared against the baseline in the table below, where P denotes patch size and S denotes stride size. The parentheses highlight the specific changes to the patch and stride sizes from baseline.

Patch Size	Stride 1	Stride 2	Stride 3	Stride 4
22				0.1042 (-0.0012)
14		0.1054 (0.0000)	0.1059 ($+0.0005$)	**0.1054 (Baseline)**
10				0.1097 ($+0.0043$)
6				0.1117 ($+0.0063$)
2	0.1077 ($+0.0023$)			
1	0.2204 ($+0.1150$)*			

*Patch 1, Stride 1 is equivalent to Raw Sequences (No Temporal Compression).

Table 2: This matrix isolates the independent and combined effects of varying patch sizes and strides on the convergent phoneme error rate, directly quantifying the performance deviation from the baseline architecture. The parentheses show the difference in PER compared to the baseline.

The experimental data indicate that bigger patch sizes generally result in a lower PER, although the improvements from increasing patch size are marginal after reaching a patch size of 14 time steps. Interestingly, there is a significant drop in performance if patching is removed entirely, as the PER increases from 0.1054 to 0.2204 when the model reads single 20ms frames at a time. Remarkably, even a minimal patch of 2 adjacent time steps (patch size=2) results in a nearly 12% reduction in validation PER compared to no patching.

The experimental data show that the stride does not impact the validation PER much, as strides of 2, 3, and 4 yielded nearly identical error rates. This indicates that as long as the patch size is sufficient to capture local trajectories, the specific overlap between patches is less important for accuracy. And using a larger stride of 4 is preferred, as it reduces the computational overhead since there will be a shorter sequence length from a bigger stride.

The physical articulation of a phoneme spans a much longer duration than the length of a time step, which is 20 ms. As noted by Bourlard and Morgan (1994), continuous speech features are highly correlated across the neighboring frames. While their observations were originally framed within the context of acoustic speech recognition, it is plausible that the researchers at UC Davis made use of the same statistical principle that there are dependencies and correlations between neighboring frames that apply to the neural data, as well as govern the muscles involved in speech.

The autocorrelation between neighboring frames serves as the inspiration for the patching and striding mechanism implemented in the BCI's architecture developed by Card et al.[3]. Historically, this approach traces back to one of the early multilayer perceptrons (MLP), NETTalk, where a window of 7 consecutive characters was used to predict the phoneme associated with the central letter (Sejnowski and Rosenberg, 1987). As traditional statistical models, such as Hidden Markov Models, when modeling for speech, often rely on the observation-independence assumption, they fail to account for the dependencies and correlations between the successive acoustic frames. To address that flaw, by providing the network a window of surrounding context, NETTalk showed that an MLP could map discrete sequences to phonemes. Even though NETTalk performed much worse than the best rule-based systems at the time, the NETTalk approach worked by learning the mapping rules automatically from examples through error-backpropagation and did not require a human to program the rules. Its ability to learn from data directly is important for domains that are not well-understood, such as neural signals, where the underlying rules for mapping noisy neural data to phonemes are not well known.

This concept of contextual frame stacking was further manifested by the Time Delay Neural Network (TDNN) (Waibel et al., 1989)[6], which was designed to classify short isolated speech sounds using continuous acoustic signals. The TDNN generates a sequence of hidden states by multiplying sliding windows of data with the same weight matrix and passing the product through an activation function to form hidden states. All those hidden states were then used to classify short speech sounds, allowing the final classification layer to evaluate the entire context with an approximation of a recurrent

memory loop. Weibel proved that a network could learn temporal patterns very well using windows of past frames.

Furthermore, in the 1994 Bourlard and Morgan experiments, there was evidence showing a strict advantage of using a contextual window over using only single frames of data for speech classification. When evaluating recognition accuracies in the speaker-independent TIMIT database, the authors compared an MLP receiving a single acoustic frame against an MLP receiving a 9-frame window. The MLP with 1 frame had a classification accuracy of 12.1%. The MLP with a 9-frame window had a classification accuracy of 26%, almost double that of the 1-frame MLP. This proves that using a window of context greatly enhances the model's ability to decipher temporal dependencies for speech data.

The experimental results shown previously demonstrate that even a minimal increase from a patch size of 1 to 2 provides a substantial performance gain. By providing just one adjacent frame, the network gains the ability to internally calculate the temporal derivative, or the rate of change between frames [1]. This allows the hidden units to extract the trajectory and velocity of neural signals to understand the "motion" of neural signals and to distinguish phonetic transitions. Instead of observing isolated voltage levels, the model can perceive the motion of the underlying neural manifold. By providing sufficient context through neighboring frames, the model offloads the burden of learning local temporal correlations at the input stage, allowing the recurrent layers to focus on capturing long-range dependencies and the high-level phonetic logic within the neural signals. Following this train of thought, the architecture in the BCI [3] implements a similar windowing mechanism on the aligned neural data.

4.6 Data Augmentation Ablations

To isolate the impact of specific augmentations, each augmentation technique was tested independently against a control configuration with no augmentation. The following table presents the convergent validation PER for these single augmentation training runs.

Configuration	**Modification**	**Convergent PER**	**Δ from Baseline**
Baseline	Full Augmentation Suite	0.1054	0.0000
Data Augmentation	No Augmentation	0.1832	+0.0778
Data Augmentation	Constant Offset Only	0.1774	+0.0720
Data Augmentation	Gaussian Smoothing Only	0.1676	+0.0622
Data Augmentation	Random Cut Only	0.1619	+0.0565
Data Augmentation	White Noise Only	0.1220	+0.0166

The experimental results demonstrate that the injection of white noise provides the most significant reduction in validation PER compared to all other augmentations, from a baseline PER of 0.1832 to 0.1220. This performance is largely attributed to the role of white noise as a regularizer to improve decoder generalizability and stability, as explained in the Card et al.[3] study. Furthermore, as noted by Bourlard and Morgan (1994)[1], injecting noise into training patterns can facilitate faster convergence and keep the system from getting stuck in suboptimal local minima.

Other data augmentations yielded very minor improvements on the validation PER. Random cutting reached a validation PER of 0.1619, as it forces the model to become invariant to the exact timing of speech intent during each trial. Gaussian smoothing reached a validation PER of 0.1676, as its primary purpose was to reduce the impact of noise on the underlying pattern in data. The constant offset augmentation resulted in a PER of 0.1774. According to Card et al.[3], this specific type of noise is added to improve robustness against non-stationary feature means, training the model to handle natural and unpredictable baseline shifts in the participant's brain signals over time. While the improvement is minor, it remains an important augmentation to maintain long-term stability in a BCI where signal means frequently drift. While each technique contributes to the stability of the system, white noise injection remains the most impactful individual augmentation for minimizing the phoneme error rate in this decoding architecture.

5 CONCLUSION

5.1 Insights

This paper provided a breakdown of an intracortical speech decoding architecture by identifying the specific components and structures that are most critical for decoding performance. Through ablations, the specific roles of temporal compression, the GRU stacks, and the data augmentations were isolated and evaluated.

Temporal compression is the most significant component for decoding accuracy. By concatenating a number of adjacent time steps into a single input vector, the model was able to focus on the trajectory of the underlying neural motion. While a GRU can theoretically learn temporal dependencies from raw, sequential inputs, the high noise floor of millisecond-level neural signals makes this practically inefficient. This mechanism, combined with a 4-step stride, balances the temporal resolution to identify

phonetic relationships with the computational efficiency needed to process long neural sequences.

The 5-layer GRU structure provides a necessary hierarchy to gain more abstract understandings of the neural signals. As indicated by the ablations, reducing this depth significantly degrades the phoneme error rate by forcing a direct mapping between the neural data and the phonemes.

5.2 Best Architecture

Following these observations, a final model is trained and tested on the blind testing set with the most effective configurations. This model has a patch size of 22 time steps with a stride of 4, with all of the original data augmentations and dropout probabilities. The number of hidden units was expanded from 768 to 1024, a modification that showed a minor improvement in the convergent PER. The convergent validation phoneme error rate of the final model is 0.1025. The final testing phoneme error rate is 0.0968. These results prove that the model is robust and that the 1024 hidden unit architecture effectively handles the decoding task.

5.3 Future Directions

Future efforts should focus on integrating attention mechanisms or Transformer-based blocks to weight specific neural features dynamically and enhance the model's performance and interpretability on decoding neural signals. Furthermore, as the original study from UC Davis [3] was limited to a single clinical participant, a future direction would be to explore the generalizability of the architecture and develop transfer learning frameworks beyond one user. This could greatly improve the scalability and accessibility of brain-computer interface systems.

REFERENCES

[1] Bourlard, Hervé and Morgan, Nelson. *Connectionist Speech Recognition: A Hybrid Approach*. Kluwer Academic Publishers, 1994. ISBN: 0-7923-9396-1.

[2] Card, Nicholas et al. *Brain-to-text '25*. 2025. URL: https://kaggle.com/competitions/brain-to-text-25.

[3] Card, Nicholas S. et al. "An Accurate and Rapidly Calibrating Speech Neuroprosthesis". In: *New England Journal of Medicine* 391.7 (Aug. 2024), pp. 609–618. DOI: 10.1056/NEJMoa2314132. URL: https://doi.org/10.1056/NEJMoa2314132 (visited on 09/20/2025).

[4] Cho, Kyunghyun et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". In: (Oct. 2014), pp. 1724–1734. DOI: 10.3115/v1/D14-1179. URL: https://arxiv.org/abs/1406.1078.

[5] Graves, Alex et al. "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks". In: (2006), pp. 369–376. URL: https://www.cs.toronto.edu/~graves/icml_2006.pdf.

[6] Waibel, Alexander et al. "Phoneme recognition using time-delay neural networks". In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 37.3 (Mar. 1989), pp. 328–339. DOI: 10.1109/29.21701. URL: https://doi.org/10.1109/29.21701.

APPENDIX 1: CTC LOSS EXAMPLE

.1 Alignment Example

The CTC function collapses a path π into a target sequence S_b by merging consecutive identical characters and removing blank tokens ($-$). The total probability $P(S_b|Y_b)$ is the sum of the probabilities of all valid alignments $\pi \in \Phi(S_b)$.

As an intuitive example, consider 280 ms (14 time steps) of brain activity intended to produce the word "HELLO". Due to variations in speech rate and neural signal timing, multiple alignments $\pi \in \Phi(S_b)$ can represent this single target sequence:

Alignment 1 (Slow speech): – – HH – HE LL OW – – $\rightarrow$ "HELLO"

Alignment 2 (Fast speech): HH EE L L OW – – – – – – $\rightarrow$ "HELLO"

Alignment 3 (Uneven timing): - HH - EE - L - L - OW - - $\rightarrow$ "HELLO"

Alignment 4 (With repetitions): HH HH EL L - L OW OW - - - $\rightarrow$ "HELLO"

.2 CTC Loss Calculation Example

Table 3 contains the model probability outputs $s^t_{b,k}$ for a simplified trial b where $T' = 2$ and the target sequence is $S_b =$ "A".

Table 3: Token Probability Distribution for $T' = 2$

Time Step	**t=1**	**t=2**
$s^t_{b,\text{-}}$ (blank)	0.5	0.4
$s^t_{b,A}$	0.4	0.3
$s^t_{b,B}$	0.1	0.3

CTC Mapping for Sequence "A"

To calculate the total probability of the sequence $S_b =$ "A", we sum the probabilities of all valid alignments π that collapse to S_b:

$$
\begin{aligned}
P(\pi = \text{"A-"}|Y_b) &= s^1_{b,A} \times s^2_{b,\text{-}} = 0.4 \times 0.4 = 0.16 \\
P(\pi = \text{"-A"}|Y_b) &= s^1_{b,\text{-}} \times s^2_{b,A} = 0.5 \times 0.3 = 0.15 \\
P(\pi = \text{"AA"}|Y_b) &= s^1_{b,A} \times s^2_{b,A} = 0.4 \times 0.3 = 0.12 \\
P(S_b = \text{"A"}|Y_b) &= P(\pi = \text{"A-"}|Y_b) + P(\pi = \text{"-A"}|Y_b) + P(\pi = \text{"AA"}|Y_b) \\
&= 0.16 + 0.15 + 0.12 = \mathbf{0.43}
\end{aligned}
$$

Full Sequence Distribution

The remaining possible sequences and their probabilities are calculated as follows:

- $P(S_b = \text{""}|Y_b) = P(\pi = \text{"- -"}|Y_b) = 0.5 \times 0.4 = 0.20$
- $P(S_b = \text{"B"}|Y_b) = P(\pi = \text{"B-"}|Y_b) + P(\pi = \text{"-B"}|Y_b) + P(\pi = \text{"BB"}|Y_b) = (0.1 \times 0.4) + (0.5 \times 0.3) + (0.1 \times 0.3) = 0.22$
- $P(S_b = \text{"AB"}|Y_b) = s^1_{b,A} \times s^2_{b,B} = 0.4 \times 0.3 = 0.12$
- $P(S_b = \text{"BA"}|Y_b) = s^1_{b,B} \times s^2_{b,A} = 0.1 \times 0.3 = 0.03$

Verification: $0.43(\text{A}) + 0.22(\text{B}) + 0.20(\text{""}) + 0.12(\text{AB}) + 0.03(\text{BA}) = 1.0$

CTC Loss Calculation

The CTC Loss for this trial is defined as the negative log likelihood of the correct target sequence. Given the ground truth sequence is $S_b =$ "A":

$$
\begin{aligned}
\mathcal{L}^{(b)}_{CTC} &= -\log P(S_b = \text{"A"}|Y_b) \\
&= -\log(0.43) \\
&\approx \mathbf{0.84}
\end{aligned}
$$